

Data Structures & Algorithms in Python

1. What is a Data Structure?

A **data structure** is a way of organizing and storing data so that it can be accessed and modified efficiently.

Example:

```
numbers = [10, 20, 30, 40, 50]
```

Here, `list` is a Python data structure.

Common Data Structures

Data Structure	Python implementation	Common use
Array	<code>list</code>	Ordered collection
Linked List	Implement using classes	Sequential data
Stack	<code>list</code> / <code>deque</code>	Undo, function calls
Queue	<code>collections.deque</code>	Waiting lines, scheduling
Hash Table	<code>dict</code> / <code>set</code>	Fast lookup
Tree	Classes / libraries	Hierarchical data
Graph	Classes / dictionaries	Networks

2. What is an Algorithm?

An **algorithm** is a sequence of steps used to solve a problem.

Example: Linear Search

```
def linear_search(arr, target):
```

```
for i in range(len(arr)):
    if arr[i] == target:
        return i
return -1
```

The algorithm checks elements one by one until it finds the target.

Data Structure vs Algorithm

Data Structure:

How do we store and organize the data?

Algorithm:

How do we process the data?

Together:

Data Structure



stores/organizes data



Algorithm



processes data



Efficient problem solving

3. Memory Allocation

When a Python program runs, it needs memory to store objects such as:

```
x = 10
name = "Rashik"
numbers = [10, 20, 30]
```

Python automatically manages memory.

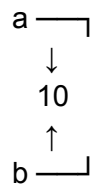
Unlike C, you normally don't manually allocate and free memory.

4. Python Objects and References

Consider:

```
a = 10  
b = a
```

Conceptually:



Both names can refer to the same object.

You can inspect an object's identity using:

```
print(id(a))  
print(id(b))
```

5. Mutable vs Immutable Objects

Immutable

An immutable object cannot be changed in place.

Examples:

```
int  
float  
str  
tuple  
bool
```

Example:

```
x = 10  
x = 20
```

The name `x` is reassigned to another object.

Mutable

A mutable object can be changed.

Examples:

```
list  
dict  
set
```

Example:

```
numbers = [10, 20, 30]  
  
numbers.append(40)
```

The list is modified.

6. Memory Management in Python

Python mainly uses:

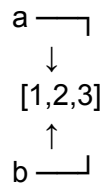
1. Reference Counting

Python keeps track of references to objects.

Example:

```
a = [1, 2, 3]  
b = a
```

Conceptually:



If:

```
del a
```

the object can still be accessed through `b`.

2. Garbage Collection

Python also has a garbage collector that helps reclaim objects that are no longer reachable, including certain reference cycles.

Example:

```
a = [1, 2, 3]
a = None
```

If no other references exist, the old list can eventually be reclaimed.

7. What is a Memory Leak?

A **memory leak** occurs when a program continues to hold memory that it no longer needs, preventing that memory from being reclaimed.

Python reduces many manual memory-management problems, but memory leaks or excessive memory retention can still happen.

Example:

```
cache = []
```

```
while True:  
    data = create_large_object()  
    cache.append(data)
```

The list keeps references to all the objects.

Therefore Python cannot reclaim them.

Memory usage keeps increasing.

Simple idea

Object created



No longer needed



Reference still exists



Cannot be reclaimed



Memory remains occupied

8. Complexity Analysis

Complexity analysis tells us how an algorithm's resource requirements grow as the input size increases.

There are two main types:

Time Complexity

How the amount of computational work grows.

Space Complexity

How the additional memory requirement grows.

9. What is n ?

n normally represents the size of the input.

Example:

```
numbers = [10, 20, 30, 40, 50]
```

There are 5 elements.

Therefore:

$n = 5$

If there are 1,000 elements:

$n = 1000$

10. Time Complexity

Example:

```
for i in range(n):  
    print(i)
```

The loop runs n times.

Therefore:

$O(n)$

This is called **linear time**.

11. $O(1)$ — Constant Time

Example:

```
arr = [10, 20, 30, 40, 50]
```

```
print(arr[2])
```

Accessing an element by index takes constant time for a Python list.

Therefore:

$O(1)$

The input can become much larger without changing the basic number of operations for that indexed access.

12. $O(n)$ — Linear Time

Example:

```
for x in arr:  
    print(x)
```

We may process every element.

Therefore:

$O(n)$

13. $O(n^2)$ — Quadratic Time

Example:

```
for i in range(n):  
    for j in range(n):  
        print(i, j)
```


The outer loop runs n times.

The inner loop also runs n times for every outer iteration.

Therefore:

$$n \times n = n^2$$

So:

$$O(n^2)$$

14. $O(\log n)$ — Logarithmic Time

Binary search repeatedly divides the search space approximately in half.

Example:

1,000,000
↓
500,000
↓
250,000
↓
125,000
↓
...
↓
1

Therefore binary search has:

$$O(\log n)$$

time complexity.

15. $O(n \log n)$

This complexity appears frequently in efficient sorting algorithms.

Examples include:

- Merge Sort
- Heap Sort
- Average-case Quick Sort

A common pattern is:

n work $\times$ $\log n$ levels

Therefore:

$O(n \log n)$

16. Big-O / Asymptotic Analysis

Big-O notation describes the growth rate of an algorithm as the input size becomes very large.

We generally ignore constants and lower-order terms.

Example 1

$O(2n)$

becomes:

$O(n)$

Example 2

$O(n^2 + n + 10)$

becomes:

$O(n^2)$

because n^2 grows faster than n .

17. Important Big-O Rules

Rule 1 — Drop constants

$$O(5n) \rightarrow O(n)$$

Rule 2 — Keep the dominant term

$$O(n^2 + n) \rightarrow O(n^2)$$

Rule 3 — Sequential operations add

for i in range(n):

...

for j in range(n):

...

$$O(n) + O(n)$$

$$= O(2n)$$

$$= O(n)$$

Rule 4 — Nested operations multiply

for i in range(n):

for j in range(n):

...

$$O(n \times n)$$

$$= O(n^2)$$

18. Big-O Hierarchy

From generally faster-growing performance to slower-growing performance:

$O(1)$
↓
 $O(\log n)$
↓
 $O(n)$
↓
 $O(n \log n)$
↓
 $O(n^2)$
↓
 $O(2^n)$
↓
 $O(n!)$

Important complexities to know:

Big-O	Name	Example
$O(1)$	Constant	List index access
$O(\log n)$	Logarithmic	Binary Search
$O(n)$	Linear	Linear Search
$O(n \log n)$	Linearithmic	Merge Sort
$O(n^2)$	Quadratic	Nested loops
$O(2^n)$	Exponential	Some recursive algorithms
$O(n!)$	Factorial	Brute-force permutations

19. Space Complexity

Space complexity measures the **additional memory required by an algorithm**.

Example:

```
def example(n):  
    total = 0
```

```
for i in range(n):  
    total += i  
  
return total
```

The loop runs n times:

Time $\rightarrow O(n)$

Only a fixed number of variables are used:

Space $\rightarrow O(1)$

20. $O(n)$ Space

Example:

```
def example(n):  
    numbers = []  
  
    for i in range(n):  
        numbers.append(i)  
  
    return numbers
```

Time:

$O(n)$

Space:

$O(n)$

because the list grows with n .

21. Recursion and Space Complexity

Example:

```
def countdown(n):  
    if n == 0:  
        return  
  
    countdown(n - 1)
```

The recursive calls build up on the call stack:

```
countdown(n)  
countdown(n-1)  
countdown(n-2)  
...  
countdown(0)
```

Therefore:

Time $\rightarrow O(n)$
Space $\rightarrow O(n)$

The extra space comes from the **call stack**.

22. Iterative vs Recursive Binary Search

Iterative Binary Search

Time $\rightarrow O(\log n)$
Space $\rightarrow O(1)$

Recursive Binary Search

Time $\rightarrow O(\log n)$
Space $\rightarrow O(\log n)$

The recursive version uses additional call-stack space.

23. Common Data Structure Complexities

Array / Python List

Operation	Typical Complexity
Access by index	$O(1)$
Search	$O(n)$
Append at end	$O(1)$ amortized
Insert at beginning	$O(n)$
Delete at beginning	$O(n)$
Delete at end	$O(1)$

Insertion/deletion in the middle is typically $O(n)$ because elements may need to be shifted.

24. Linked List

For a typical singly linked list:

Operation	Complexity
Access	$O(n)$
Search	$O(n)$
Insert at head	$O(1)$
Delete at head	$O(1)$
Insert after known node	$O(1)$

Delete at known node/reference **$O(1)$**

If you first have to search for the node:

Search $\rightarrow O(n)$

Insertion/Deletion $\rightarrow O(1)$

Overall $\rightarrow O(n)$

25. Stack

A stack follows:

LIFO — Last In, First Out

Example:

```
stack = []
```

```
stack.append(10)
```

```
stack.append(20)
```

```
stack.append(30)
```

```
stack.pop()
```

Typical operations:

Operation	Complexity
Push	$O(1)$ amortized
Pop	$O(1)$
Peek/top	$O(1)$
Search	$O(n)$

26. Queue

A queue follows:

FIFO — First In, First Out

Use `collections.deque` in Python:

```
from collections import deque
```

```
queue = deque()
```

```
queue.append(10)
queue.append(20)
queue.append(30)
```

```
queue.popleft()
```

Typical operations:

Operation	Complexity
Enqueue	$O(1)$
Dequeue	$O(1)$
Front/peek	$O(1)$
Search	$O(n)$

Avoid using `list.pop(0)` for an efficient queue because it is $O(n)$.

27. Hash Table

Python's:

`dict`

and:

set

are hash-table-based structures.

Example:

```
student = {  
    "name": "Ali",  
    "age": 18  
}
```

```
print(student["name"])
```

Average-case complexity:

Operation	Average
Lookup	$O(1)$
Insert	$O(1)$
Delete	$O(1)$

Worst-case behavior can be worse.

28. Tree

Example Binary Search Tree:

```
    50  
   /  \  
  30   70  
 /  \ /  \  
20 40 60 80
```

For a balanced BST:

Operation	Average
Search	$O(\log n)$
Insert	$O(\log n)$
Delete	$O(\log n)$

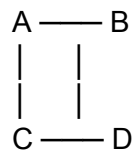
A badly unbalanced BST can degrade to:

$O(n)$

29. Graph

A graph represents relationships between objects.

Example:



Two common representations:

Adjacency Matrix

Space:

$O(V^2)$

Adjacency List

Space:

$O(V + E)$

Where:

V = number of vertices
E = number of edges

30. Quick Revision

Data Structure

A way of organizing and storing data efficiently.

Algorithm

A step-by-step procedure for solving a problem.

Memory Allocation

Providing memory for objects/data used by a program.

Memory Leak

Memory that remains occupied because a program unnecessarily retains references to objects that are no longer needed.

Time Complexity

How an algorithm's computational work grows with input size.

Space Complexity

How an algorithm's additional memory requirement grows with input size.

Big-O

A notation used to describe the asymptotic growth of an algorithm's resource usage as input size increases.

Most Important Things to Memorize

$O(1)$ → Constant

$O(\log n)$ → Binary Search
 $O(n)$ → Linear Search
 $O(n \log n)$ → Efficient Sorting
 $O(n^2)$ → Nested Loops

And remember:

Sequential → ADD

Nested → MULTIPLY

Constants → DROP

Smaller terms → DROP

For example:

$O(2n + 10)$ → $O(n)$

$O(n^2 + n)$ → $O(n^2)$

$O(n) + O(n)$ → $O(n)$

$O(n) \times O(n)$ → $O(n^2)$

Arrays in Python — Complete Notes

1. What is an Array?

An **array** is a data structure used to store multiple values in a single collection.

The elements are stored in an **ordered sequence** and can be accessed using an **index**.

In Python, the most commonly used array-like data structure is the **list**.

```
numbers = [10, 20, 30, 40, 50]
```

The indexes are:

Value:	10	20	30	40	50
Index:	0	1	2	3	4

Python also has an `array` module, but for general DSA learning, Python `list` is what you'll most often use.

2. Creating an Array/List

```
numbers = [10, 20, 30, 40, 50]
```

You can also create an empty list:

```
numbers = []
```

And add values:

```
numbers.append(10)
numbers.append(20)
numbers.append(30)
```

Result:

```
[10, 20, 30]
```

3. Accessing Elements

You access an element using its index.

```
numbers = [10, 20, 30, 40, 50]
```

```
print(numbers[0])
print(numbers[2])
```

Output:

10
30

Remember:

Python indexes start from 0.

Index:	0	1	2	3	4
	↓	↓	↓	↓	↓
Array:	10	20	30	40	50

Time Complexity

Accessing by index:

$O(1)$

Why?

Because Python can directly access the required position rather than checking every element.

4. Updating an Element

```
numbers = [10, 20, 30, 40]
```

```
numbers[2] = 100
```

```
print(numbers)
```

Output:

```
[10, 20, 100, 40]
```

Updating by index is:

$O(1)$

5. Searching

Suppose:

```
numbers = [10, 20, 30, 40, 50]
```

```
print(40 in numbers)
```

Python may have to check elements one by one:

```
10   
20   
30   
40 
```

Therefore, searching an unsorted list is typically:

$O(n)$

6. Inserting an Element

You can add an element to the end:

```
numbers.append(60)
```

Typical/amortized complexity:

$O(1)$

But inserting at the beginning:

```
numbers.insert(0, 5)
```

requires existing elements to shift:

Before:

```
[10, 20, 30, 40]
```


After:

[5, 10, 20, 30, 40]

Therefore:

$O(n)$

7. Deleting an Element

Delete the last element:

`numbers.pop()`

Typical complexity:

$O(1)$

Delete the first element:

`numbers.pop(0)`

Elements have to shift:

Before:

[10, 20, 30, 40]

After:

[20, 30, 40]

Therefore:

$O(n)$

8. Traversing an Array

Traversal means visiting every element.

```
numbers = [10, 20, 30, 40, 50]
```

```
for number in numbers:  
    print(number)
```

If there are n elements, we visit n elements.

Therefore:

Time $\rightarrow O(n)$

9. Common Array/List Operations

Operation	Python	Typical Time
Access	<code>arr[i]</code>	$O(1)$
Update	<code>arr[i] = x</code>	$O(1)$
Search	<code>x in arr</code>	$O(n)$
Traverse	<code>for x in arr</code>	$O(n)$
Append	<code>arr.append(x)</code>	$O(1)$ amortized
Insert beginning	<code>arr.insert(0, x)</code>	$O(n)$
Delete end	<code>arr.pop()</code>	$O(1)$
Delete beginning	<code>arr.pop(0)</code>	$O(n)$

This table is **very important for your DSA assignment.**

10. Use Cases of Arrays

Arrays/lists are useful when you need to store a collection of related values.

1. Student Marks

```
marks = [85, 90, 78, 92, 88]
```

2. Employee Salaries

```
salaries = [30000, 45000, 50000, 60000]
```

3. Product Prices

```
prices = [100, 250, 500, 750]
```

4. Storing Usernames

```
users = ["Ali", "John", "Rahul", "Sara"]
```

5. Processing Data

```
numbers = [10, 20, 30, 40, 50]
```

```
total = 0
```

```
for number in numbers:  
    total += number
```

```
print(total)
```

6. Implementing Other Data Structures

Python lists can be used to implement things such as:

Stack

Dynamic arrays

Some graph representations

11. Advantages of Arrays

1. Fast Access

You can access an element directly:

```
arr[500]
```

Typical complexity:

$O(1)$

2. Simple to Use

Python lists are easy to create and manipulate:

```
numbers = [10, 20, 30]
```

3. Good for Sequential Data

If your data naturally forms a sequence:

10 → 20 → 30 → 40 → 50

a list is often a good choice.

4. Efficient Iteration

You can easily process every element:

```
for x in numbers:  
    print(x)
```

5. Dynamic Size in Python

Python lists can grow and shrink:

```
numbers = []
```

```
numbers.append(10)  
numbers.append(20)  
numbers.append(30)
```

Unlike a traditional fixed-size array in languages such as C, Python lists automatically resize as needed.

12. Disadvantages of Arrays

1. Insertion at Beginning/Middle Can Be Expensive

```
numbers.insert(0, 10)
```

Elements may need to shift.

Complexity:

$O(n)$

2. Deletion at Beginning/Middle Can Be Expensive

```
numbers.pop(0)
```

Again, elements may need to shift.

$O(n)$

3. Searching an Unsorted Array is Slow

For:

```
numbers = [10, 20, 30, 40, 50]
```

finding 50 may require checking every element.

$O(n)$

4. Wasted Capacity Can Exist

Python lists are dynamic arrays and may allocate additional capacity internally so that repeated appends are efficient.

That means the underlying allocated memory can temporarily be larger than the number of elements currently stored.

5. Not Always the Best Structure

If your application requires extremely frequent insertions/deletions at the beginning, a different data structure such as a deque may be more appropriate.

13. Array vs Linked List

This is a very important comparison.

Feature	Array/List	Linked List
Access	$O(1)$	$O(n)$
Search	$O(n)$	$O(n)$
Insert at beginning	$O(n)$	$O(1)$
Delete at beginning	$O(n)$	$O(1)$
Memory layout	Contiguous dynamic-array storage	Nodes connected by references
Easy random access	✓	✗

So:

Choose an array/list when you need fast index access.

A linked list can be useful when you frequently insert/delete nodes and already have the relevant node/reference.

14. Real-World Example

Imagine you're storing the marks of 10,000 students:

marks = [78, 85, 91, 67, 88, ...]

You want:

"Give me the marks of student at position 5000."

You can do:

marks[5000]

Typical complexity:

$O(1)$

That's one of the biggest strengths of arrays/lists.

But if you repeatedly insert elements at the beginning:

marks.insert(0, new_mark)

that's much more expensive:

$O(n)$

15. What You Should Remember

Definition

An array is a data structure that stores elements in an ordered sequence and provides indexed access.

For Python DSA, you'll mainly work with **lists as dynamic arrays**.

Advantages

Fast index access $\rightarrow O(1)$
Easy to use
Good for sequential data
Efficient iteration
Dynamic resizing in Python lists

Disadvantages

Insertion at beginning/middle $\rightarrow O(n)$
Deletion at beginning/middle $\rightarrow O(n)$
Searching unsorted data $\rightarrow O(n)$
May allocate extra capacity internally
Not ideal for every type of workload

Most important complexity table

Access $\rightarrow O(1)$
Update $\rightarrow O(1)$
Search $\rightarrow O(n)$
Traversal $\rightarrow O(n)$
Append $\rightarrow O(1)$ amortized
Insert front $\rightarrow O(n)$
Delete end $\rightarrow O(1)$
Delete front $\rightarrow O(n)$

Linked List in Python — Complete Notes

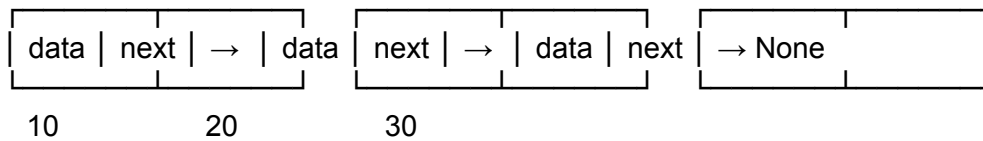
1. What is a Linked List?

A **linked list** is a linear data structure where elements are stored in separate objects called **nodes**.

Each node contains:

1. **Data** — the value stored in the node
2. **Next** — a reference to the next node

Conceptually:



A linked list starts from a node called the **head**.

Head



10 → 20 → 30 → 40 → None

Unlike a Python list, a linked list does not provide direct indexed access.

2. Creating a Node in Python

Python does not have a built-in `LinkedList` class like some languages.

We can create one ourselves using a class.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

Create nodes:

```
node1 = Node(10)
node2 = Node(20)
node3 = Node(30)
```

Connect them:

```
node1.next = node2
node2.next = node3
```

Now we have:

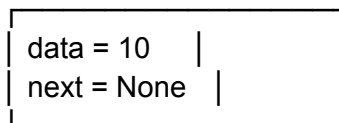
10 → 20 → 30 → None

3. Understanding the Node

Consider:

```
node = Node(10)
```

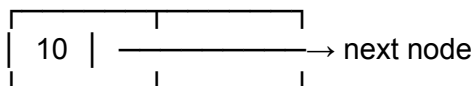
The node contains:



When we connect it:

```
node.next = another_node
```

it becomes:



The **next** variable stores a **reference to another node**.

4. What is the Head?

The **head** is the first node of the linked list.

Example:

Head



10 → 20 → 30 → 40 → None

In Python:

```
head = node1
```

The **head** gives us access to the beginning of the list.

5. Traversing a Linked List

Traversal means visiting each node one by one.

```
current = head
```

```
while current is not None:  
    print(current.data)  
    current = current.next
```

Output:

10
20
30
40

The process is:

Head

↓

10 → 20 → 30 → 40 → None

↑

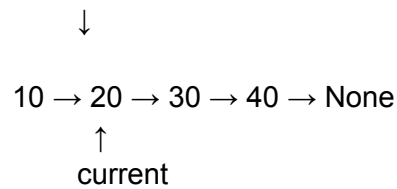
current

↓

10 → 20 → 30 → 40 → None

↑

current



Since we may visit every node:

Time → $O(n)$

6. Accessing an Element

Suppose:

10 → 20 → 30 → 40 → 50

You want the 5th element.

With an array/list:

arr[4]

can directly access the position.

But with a linked list, we must start from the head:

10 → 20 → 30 → 40 → 50

↑
start

Then move:

10 → 20 → 30 → 40 → 50

↑

Then:

10 → 20 → 30 → 40 → 50
↑

And so on.

Therefore:

Access → $O(n)$

Important

A linked list does **not** support efficient random/index access.

7. Searching

Suppose we want to find 40:

10 → 20 → 30 → 40 → 50

We may have to check:

10 ✗
20 ✗
30 ✗
40 ✓

Therefore:

Search → $O(n)$

8. Inserting at the Beginning

Suppose:

10 → 20 → 30

We want to insert 5 at the beginning.

Create a new node:

5

Make it point to the old head:

5 → 10 → 20 → 30

Then make 5 the new head.

In Python:

```
new_node = Node(5)
```

```
new_node.next = head
```

```
head = new_node
```

Only a few references need to be changed.

Therefore:

Insert at beginning → O(1)

9. Deleting from the Beginning

Suppose:

Head

↓

10 → 20 → 30

We want to remove 10.

Simply move the head:

`head = head.next`

Now:

Head

↓

`20 → 30`

Therefore:

Delete from beginning → $O(1)$

10. Inserting After a Known Node

Suppose:

`10 → 20 → 40`

We already have a reference to node `20`.

We want to insert `30`.

Create:

`new_node = Node(30)`

Then:

`new_node.next = node20.next`

`node20.next = new_node`

Result:

`10 → 20 → 30 → 40`

Only a few references are changed.

Therefore:

Insertion after a known node $\rightarrow O(1)$

Important condition

We must **already have the node/reference**.

If we first have to search for node 20:

Search $\rightarrow O(n)$

Insert $\rightarrow O(1)$

Overall $\rightarrow O(n)$

11. Deleting a Node

Suppose:

10 $\rightarrow$ 20 $\rightarrow$ 30 $\rightarrow$ 40

We want to delete 30.

We need to connect 20 directly to 40:

10 $\rightarrow$ 20 $\longrightarrow$ 40

If we already have the necessary reference to the previous node:

`previous.next = current.next`

the pointer change itself is:

$O(1)$

But if we first need to search for the node:

Search $\rightarrow O(n)$
Deletion $\rightarrow O(1)$

Overall $\rightarrow O(n)$

12. Types of Linked Lists

There are three important types.

1. Singly Linked List

Each node points to the next node.

10 $\rightarrow$ 20 $\rightarrow$ 30 $\rightarrow$ None

Each node has:

data
next

2. Doubly Linked List

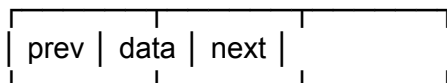
Each node has two references:

- prev
- next

Example:

None $\leftarrow$ 10 $\rightleftarrows$ 20 $\rightleftarrows$ 30 $\rightarrow$ None

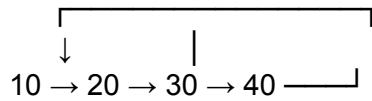
Conceptually:



This allows movement in both directions.

3. Circular Linked List

The last node points back to the first node.



There is no **None** at the end.

13. Advantages of Linked Lists

1. Efficient Insertion

If you already have the required node/reference, insertion can be:

$O(1)$

especially at the head or after a known node.

2. Efficient Deletion

Deleting from the beginning is:

$O(1)$

Deletion can also be $O(1)$ when the required position/reference information is already available.

3. Dynamic Size

A linked list can grow and shrink as needed.

You don't need to specify a fixed capacity beforehand.

4. No Element Shifting

In an array/list:

[10, 20, 30, 40]

inserting at the beginning may require shifting elements.

In a linked list:

10 → 20 → 30

we can change references instead of shifting all the existing elements.

14. Disadvantages of Linked Lists

1. Slow Access

You cannot directly do something equivalent to:

list[500]

in $O(1)$.

You must traverse from the head.

Therefore:

Access → $O(n)$

2. Extra Memory

Every node needs to store a reference such as `next`.

A doubly linked list needs both:

```
prev  
next
```

Therefore, linked lists require additional memory for these references.

3. Poorer Cache Locality

Linked-list nodes can be scattered in memory rather than stored in one contiguous block.

This can make traversal less cache-friendly than array-based storage.

4. More Complex

Arrays/lists are simple:

```
numbers = [10, 20, 30]
```

A linked list requires nodes and references:

```
class Node:  
    ...
```

Therefore, linked lists require more implementation logic.

5. Searching is Still $O(n)$

A linked list does not automatically make searching faster.

For an ordinary unsorted linked list:

Search $\rightarrow O(n)$

15. Linked List vs Array/List

This is one of the most important comparisons.

Operation	Array/List	Linked List
Access by index	$O(1)$	$O(n)$
Search	$O(n)$	$O(n)$
Insert at beginning	$O(n)$	$O(1)$
Delete at beginning	$O(n)$	$O(1)$
Insert after known position/node	$O(n)^*$	$O(1)$
Delete with known reference/position	$O(n)^*$	$O(1)$
Extra pointer/reference memory	Low	Higher

* Array/list insertion or deletion may require shifting elements.

16. Time Complexity Summary

Singly Linked List

Operation	Complexity
Access	$O(n)$
Search	$O(n)$
Insert at head	$O(1)$
Delete at head	$O(1)$

Insert after known node $O(1)$

Delete with required
reference $O(1)$

Traverse $O(n)$

Important Condition

When you see:

"Insert/delete at a known node"

think:

$O(1)$

When you see:

"Find the node first"

think:

Search $\rightarrow O(n)$

Therefore the overall operation may be:

$O(n)$

17. Real-World Use Cases

Linked lists can be useful when data changes frequently and insertion/deletion operations are important.

Examples include:

1. Music Playlist

Song 1 $\rightarrow$ Song 2 $\rightarrow$ Song 3 $\rightarrow$ Song 4

Songs can be linked together.

2. Browser Navigation

A doubly linked structure can conceptually represent:

Previous ⇌ Current ⇌ Next

3. Implementing Other Data Structures

Linked lists can be used to implement:

- Stacks
- Queues
- Hash table chaining

4. Memory/Resource Management

Linked structures can be useful in some systems-level data structures where dynamically changing collections are needed.

18. Simple Complete Linked List Example

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

```
# Create nodes
node1 = Node(10)
node2 = Node(20)
node3 = Node(30)
```

```
# Connect nodes
node1.next = node2
node2.next = node3
```

```
# Head
head = node1
```

```
# Traverse
current = head

while current is not None:
    print(current.data)
    current = current.next
```

Output:

```
10
20
30
```

The structure is:

```
Head
↓
10 → 20 → 30 → None
```

19. What You Should Remember

Definition

A linked list is a linear data structure made up of nodes, where each node stores data and a reference to another node.

Main Parts

```
Node
├── data
└── next
```

Advantages

- Dynamic size
- Efficient insertion/deletion when position/reference is known
- No shifting of existing elements
- Useful for implementing other structures

Disadvantages

Slow random access $\rightarrow O(n)$
Extra memory for references
Poorer cache locality
More complex than arrays/lists
Search $\rightarrow O(n)$

Most Important Complexities

Access $\rightarrow O(n)$
Search $\rightarrow O(n)$
Insert at head $\rightarrow O(1)$
Delete at head $\rightarrow O(1)$
Insert after known $\rightarrow O(1)$
Delete with known reference $\rightarrow O(1)$
Traversal $\rightarrow O(n)$

The key idea

ARRAY/LIST

Fast access

↓

$O(1)$ access

LINKED LIST

Fast linking/unlinking when position is known

↓

$O(1)$ insertion/deletion

↓

But $O(n)$ access

Core rule to remember:

Arrays are good when you need fast access. Linked lists are good when you need efficient insertion/deletion at known positions.